

SDD — Módulo 3: Grafos

1. Información general del proyecto

Nombre del proyecto: Visualizador Web de Estructuras de Datos

Módulo: Grafos

Versión: 0.1.0

Estado: Especificación inicial para implementación del tercer módulo

Tecnologías: Python, Flask, HTML5, CSS3, JavaScript, Jinja2

Tipo de proyecto: Aplicación web didáctica

Público objetivo: Estudiantes, docentes y monitores de estructuras de datos

Descripción general

El módulo de grafos permitirá a los estudiantes construir, modificar, recorrer y analizar grafos dirigidos y no dirigidos, ponderados y no ponderados, mediante una interfaz web visual e interactiva.

Este módulo debe integrarse con la arquitectura general del proyecto y conservar las mismas especificaciones de calidad, documentación, separación de capas, ayudas didácticas, trazabilidad, pruebas y criterios de aceptación definidos para los módulos anteriores.

El módulo también incluirá **Union-Find** como estructura auxiliar para comprender conjuntos disjuntos y como soporte conceptual del algoritmo de Kruskal.

Objetivo del módulo

Diseñar e implementar un módulo web didáctico que permita crear grafos, ejecutar operaciones básicas, visualizar recorridos y analizar algoritmos como BFS, DFS, Dijkstra, Bellman-Ford, Prim y Kruskal.

Alcance inicial

La primera versión del módulo de grafos debe incluir:

- Grafo dirigido.
- Grafo no dirigido.
- Grafo ponderado.
- Grafo no ponderado.
- Recorridos BFS y DFS.
- Caminos mínimos con Dijkstra.
- Caminos mínimos con Bellman-Ford.
- Árbol de expansión mínima con Prim.
- Árbol de expansión mínima con Kruskal.
- Union-Find como estructura auxiliar.
- Excepciones comunes reutilizadas desde el dominio.

Fuera de alcance para esta fase

- Visualización 3D.

- Persistencia en base de datos.
- Autenticación de usuarios.
- Evaluación automática de estudiantes.
- Edición colaborativa en tiempo real.
- Implementaciones en C, aunque la arquitectura debe quedar preparada para ellas.
- Algoritmos avanzados no presentes en el TAD, como Floyd-Warshall, A*, Kosaraju o Tarjan, salvo que sean marcados como pendientes futuros.

2. Visión del sistema

Problema que resuelve

Los grafos suelen ser complejos para estudiantes porque combinan vértices, aristas, direccionalidad, pesos, recorridos, caminos mínimos y árboles de expansión. El módulo busca hacer visibles esos conceptos mediante interacción, animaciones y mensajes didácticos.

Usuarios objetivo

- Estudiantes de estructuras de datos.
- Estudiantes de algoritmos.
- Docentes de programación.
- Monitores académicos.
- Desarrolladores educativos.

Beneficios esperados

- Comprensión visual de grafos dirigidos y no dirigidos.
- Comprensión de grafos ponderados y no ponderados.
- Visualización de recorridos BFS y DFS.
- Visualización de caminos mínimos.
- Visualización de árboles de expansión mínima.
- Comprensión de Union-Find como soporte de Kruskal.
- Base extensible para algoritmos futuros.
- Preparación para futuras implementaciones en C.

Contexto de uso

El módulo se usará desde un navegador web dentro de la misma aplicación Flask del proyecto principal.

3. Requisitos funcionales

ID	Requisito	Descripción	Prioridad	Actor
RF-G-001	Acceso al módulo	Permitir ingresar al módulo de grafos desde la página principal	Alta	Estudiante
RF-G-002	Crear grafo	Permitir crear un grafo dirigido o no dirigido	Alta	Estudiante

ID	Requisito	Descripción	Prioridad	Actor
RF-G-003	Configurar ponderación	Permitir trabajar con aristas con peso o peso por defecto	Alta	Estudiante
RF-G-004	Insertar vértice	Permitir agregar vértices al grafo	Alta	Estudiante
RF-G-005	Eliminar vértice	Permitir eliminar vértices y sus aristas asociadas	Alta	Estudiante
RF-G-006	Insertar arista	Permitir agregar aristas entre vértices, con peso opcional	Alta	Estudiante
RF-G-007	Eliminar arista	Permitir eliminar aristas existentes	Alta	Estudiante
RF-G-008	Consultar vértices	Mostrar lista de vértices existentes	Media	Estudiante
RF-G-009	Consultar aristas	Mostrar lista de aristas existentes	Media	Estudiante
RF-G-010	Consultar vecinos	Mostrar vecinos de un vértice	Media	Estudiante
RF-G-011	Consultar peso	Mostrar peso de una arista	Media	Estudiante
RF-G-012	BFS	Ejecutar recorrido en anchura desde un vértice inicial	Alta	Estudiante
RF-G-013	DFS	Ejecutar recorrido en profundidad desde un vértice inicial	Alta	Estudiante
RF-G-014	Dijkstra	Ejecutar caminos mínimos con pesos no negativos	Alta	Estudiante
RF-G-015	Bellman-Ford	Ejecutar caminos mínimos permitiendo pesos negativos y detectar ciclos negativos	Alta	Estudiante
RF-G-016	Prim	Ejecutar árbol de expansión mínima en grafos no dirigidos	Alta	Estudiante
RF-G-017	Kruskal	Ejecutar árbol de expansión mínima en grafos no dirigidos usando Union-Find	Alta	Estudiante
RF-G-018	Visualización de grafo	Mostrar vértices, aristas, pesos y dirección	Alta	Estudiante
RF-G-019	Resaltado de algoritmos	Resaltar nodos y aristas involucrados en recorridos y resultados	Alta	Estudiante
RF-G-020	Visualización de Union-Find	Mostrar conjuntos, representantes y uniones durante Kruskal	Media	Estudiante

ID	Requisito	Descripción	Prioridad	Actor
RF-G-021	Ayuda del módulo	Proveer ayuda general sobre grafos	Alta	Estudiante
RF-G-022	Ayuda por algoritmo	Proveer ayuda para BFS, DFS, Dijkstra, Bellman-Ford, Prim y Kruskal	Alta	Estudiante
RF-G-023	Manejo de errores	Traducir excepciones técnicas a mensajes didácticos	Alta	Estudiante
RF-G-024	Estado de sesión	Mantener temporalmente el estado del grafo activo	Media	Estudiante
RF-G-025	Separación lógica/visual	Separar TAD, adaptadores, servicios, rutas, plantillas y JavaScript	Alta	Desarrollador
RF-G-026	Preparación para C	Mantener contratos de adaptadores compatibles con futuras implementaciones en C	Alta	Desarrollador
RF-G-027	Plantillas reutilizables	Usar el mismo header, footer, navbar y layout base del proyecto	Alta	Desarrollador

4. Requisitos no funcionales

ID	Tipo	Descripción
RNF-G-001	Documentación	Todo el código debe estar documentado según estándares del lenguaje
RNF-G-002	Python	El código Python debe cumplir PEP8, usar docstrings y type hints cuando sea razonable
RNF-G-003	JavaScript	Separar eventos, renderizado, animaciones y comunicación con backend
RNF-G-004	HTML	Usar HTML semántico y plantillas Jinja2 reutilizables
RNF-G-005	CSS	Separar estilos base, layout, componentes y visualización de grafos
RNF-G-006	Modularidad	Grafo y Union-Find deben estar encapsulados mediante adaptadores
RNF-G-007	Mantenibilidad	Debe ser sencillo agregar nuevos algoritmos de grafos
RNF-G-008	Seguridad	Validar entradas y evitar ejecución de código arbitrario
RNF-G-009	Usabilidad	La interfaz debe ser clara para estudiantes principiantes

ID	Tipo	Descripción
RNF-G-010	Rendimiento	El sistema debe funcionar con grafos pequeños y medianos de uso didáctico
RNF-G-011	Extensibilidad	La integración futura con C no debe afectar la interfaz web
RNF-G-012	Trazabilidad	Cada requisito debe mapearse con casos de uso, módulos y pruebas
RNF-G-013	Accesibilidad	Usar colores, etiquetas y textos alternativos para reforzar comprensión
RNF-G-014	Consistencia visual	Mantener el mismo sistema visual de los módulos anteriores
RNF-G-015	Claridad algorítmica	Los resultados de algoritmos deben mostrarse con explicación paso a paso cuando sea posible

5. Casos de uso

CU-G-001 — Acceder al módulo de grafos

Actor: Estudiante

Descripción: El usuario ingresa al módulo de grafos.

Precondiciones: La aplicación principal está disponible.

Flujo principal:

1. El usuario abre la página principal.
2. Selecciona "Grafos".
3. El sistema muestra la página principal del módulo.
4. El sistema muestra opciones para crear o cargar un grafo temporal.

Postcondición: El usuario puede crear o modificar un grafo.

CU-G-002 — Crear grafo

Actor: Estudiante

Descripción: El usuario crea un grafo dirigido o no dirigido.

Precondiciones: El usuario está en el módulo de grafos.

Flujo principal:

1. El usuario selecciona si el grafo será dirigido o no dirigido.
2. El usuario confirma la creación.
3. El sistema crea el grafo en sesión.
4. El sistema muestra el área visual vacía.

Postcondición: Existe un grafo activo.

CU-G-003 — Insertar vértice

Actor: Estudiante

Descripción: El usuario agrega un vértice al grafo.

Precondiciones: Existe un grafo activo.

Flujo principal:

1. El usuario ingresa el identificador del vértice.
2. El sistema valida que el dato sea utilizable como etiqueta.
3. El adaptador invoca `insertar_vertice`.
4. El sistema actualiza la visualización.

Postcondición: El vértice queda agregado al grafo.

CU-G-004 — Insertar arista

Actor: Estudiante

Descripción: El usuario agrega una arista entre dos vértices.

Precondiciones: Existe un grafo activo.

Flujo principal:

1. El usuario ingresa origen, destino y peso opcional.
2. El sistema valida los datos.
3. El adaptador invoca `insertar_arista`.
4. Si los vértices no existen, el TAD puede crearlos automáticamente.
5. El sistema actualiza la visualización.

Postcondición: La arista queda agregada al grafo.

CU-G-005 — Eliminar vértice

Actor: Estudiante

Descripción: El usuario elimina un vértice del grafo.

Precondiciones: Existe un grafo activo.

Flujo principal:

1. El usuario ingresa el vértice a eliminar.
2. El sistema valida la entrada.
3. El adaptador invoca `eliminar_vertice`.
4. El sistema elimina visualmente el vértice y sus aristas asociadas.

Flujo alternativo: Si el vértice no existe, el sistema muestra mensaje didáctico.

Postcondición: El vértice queda eliminado si existía.

CU-G-006 — Eliminar arista

Actor: Estudiante

Descripción: El usuario elimina una arista del grafo.

Precondiciones: Existe un grafo activo.

Flujo principal:

1. El usuario ingresa origen y destino.
2. El adaptador invoca `eliminar_arista`.
3. El sistema actualiza la visualización.

Postcondición: La arista queda eliminada si existía.

CU-G-007 — Ejecutar BFS

Actor: Estudiante

Descripción: El usuario ejecuta recorrido en anchura.

Precondiciones: El vértice inicial existe.

Flujo principal:

1. El usuario ingresa vértice inicial.
2. El sistema ejecuta `bfs`.
3. El sistema muestra el orden de visita.
4. El sistema resalta los vértices recorridos.
5. El sistema explica el uso conceptual de cola.

Postcondición: El usuario visualiza el recorrido BFS.

CU-G-008 — Ejecutar DFS

Actor: Estudiante

Descripción: El usuario ejecuta recorrido en profundidad.

Precondiciones: El vértice inicial existe.

Flujo principal:

1. El usuario ingresa vértice inicial.
2. El sistema ejecuta `dfs`.
3. El sistema muestra el orden de visita.
4. El sistema resalta los vértices recorridos.
5. El sistema explica el uso conceptual de pila.

Postcondición: El usuario visualiza el recorrido DFS.

CU-G-009 — Ejecutar Dijkstra

Actor: Estudiante

Descripción: El usuario calcula caminos mínimos desde un vértice inicial con pesos no negativos.

Precondiciones: El vértice inicial existe y no hay pesos negativos.

Flujo principal:

1. El usuario ingresa vértice inicial.
2. El sistema valida que el grafo no tenga pesos negativos.
3. El sistema ejecuta `dijkstra`.
4. El sistema muestra distancias y predecesores.
5. El sistema resalta caminos resultantes cuando sea posible.

Flujo alternativo: Si hay pesos negativos, se muestra mensaje didáctico indicando que Dijkstra no aplica.

Postcondición: El usuario visualiza distancias mínimas y predecesores.

CU-G-010 — Ejecutar Bellman-Ford

Actor: Estudiante

Descripción: El usuario calcula caminos mínimos permitiendo pesos negativos.

Precondiciones: El vértice inicial existe.

Flujo principal:

1. El usuario ingresa vértice inicial.
2. El sistema ejecuta `bellman_ford`.
3. El sistema muestra distancias y predecesores.
4. El sistema indica si existe ciclo negativo.

Postcondición: El usuario visualiza el resultado del algoritmo y la existencia o no de ciclo negativo.

CU-G-011 — Ejecutar Prim

Actor: Estudiante

Descripción: El usuario calcula un árbol de expansión mínima usando Prim.

Precondiciones: El grafo es no dirigido.

Flujo principal:

1. El usuario selecciona Prim.
2. Opcionalmente ingresa un vértice inicial.
3. El sistema ejecuta `prim`.
4. El sistema muestra aristas seleccionadas y peso total.
5. El sistema resalta el árbol resultante.

Flujo alternativo: Si el grafo es dirigido, se muestra mensaje didáctico.

Postcondición: El usuario visualiza el árbol de expansión mínima.

CU-G-012 — Ejecutar Kruskal

Actor: Estudiante

Descripción: El usuario calcula un árbol de expansión mínima usando Kruskal.

Precondiciones: El grafo es no dirigido.

Flujo principal:

1. El usuario selecciona Kruskal.
2. El sistema ejecuta `kruskal`.
3. El sistema muestra aristas seleccionadas y peso total.
4. El sistema muestra explicación de conjuntos disjuntos.
5. Si es posible, muestra estado conceptual de Union-Find.

Flujo alternativo: Si el grafo es dirigido, se muestra mensaje didáctico.

Postcondición: El usuario visualiza el árbol de expansión mínima y la función de Union-Find.

CU-G-013 — Consultar ayuda

Actor: Estudiante

Descripción: El usuario consulta ayuda del módulo o de un algoritmo.

Precondiciones: El usuario está en la aplicación.

Flujo principal:

1. El usuario selecciona ayuda.
2. El sistema muestra definición conceptual.
3. El sistema muestra operaciones principales.
4. El sistema muestra complejidad, ejemplos y errores comunes.

Postcondición: El usuario obtiene apoyo didáctico.

6. Modelo de datos

6.1 Diccionario de datos

Entidad: GraphSession

Atributo	Tipo	Descripción
session_id	string	Identificador de sesión
module	string	Valor fijo: graphs
graph_type	string	directed o undirected
weighted	bool	Indica si el grafo usa pesos explícitos
state	dict/list	Estado serializable del grafo
last_operation	string	Última operación ejecutada

Atributo	Tipo	Descripción
last_result	dict/string	Resultado de la última operación
errors	list	Errores recientes

Entidad: GraphVisualState

Atributo	Tipo	Descripción
directed	bool	Indica si el grafo es dirigido
weighted	bool	Indica si las aristas muestran pesos
nodes	list	Lista de nodos visuales
edges	list	Lista de aristas visuales
metadata	dict	Cantidad de vértices, aristas y tipo de grafo
algorithm_result	dict/null	Resultado del último algoritmo
highlighted_nodes	list	Vértices resaltados
highlighted_edges	list	Aristas resaltadas

Entidad: GraphNodeVisualState

Atributo	Tipo	Descripción
id	string	Identificador único del nodo
label	string	Etiqueta visible del vértice
x	number/null	Posición opcional en eje X
y	number/null	Posición opcional en eje Y
highlighted	bool	Indica si el nodo está resaltado
visited_order	int/null	Orden de visita en recorridos

Entidad: GraphEdgeVisualState

Atributo	Tipo	Descripción
id	string	Identificador único de la arista
source	string	Vértice origen
target	string	Vértice destino
weight	number	Peso de la arista
directed	bool	Indica si debe dibujarse flecha
highlighted	bool	Indica si la arista está resaltada

Entidad: UnionFindVisualState

Atributo	Tipo	Descripción
sets	dict	Representación de conjuntos por representante
parents	dict	Mapa de padres internos
ranks	dict	Mapa de rangos internos
last_union	list/null	Última unión realizada
connected	bool/null	Resultado de última consulta de conexión

7. Arquitectura del sistema

7.1 Estilo arquitectónico

El módulo de grafos debe seguir la misma arquitectura por capas del proyecto:

```
Usuario
  ↓
Frontend HTML/CSS/JavaScript
  ↓
Rutas Flask / Controladores
  ↓
Servicios de aplicación
  ↓
Adaptadores de grafos y Union-Find
  ↓
TAD Python actuales
  ↓
Futuras implementaciones en C
```

7.2 Capas principales

Capa de presentación

Responsable de:

- Renderizar plantillas del módulo de grafos.
- Dibujar vértices y aristas.
- Mostrar flechas en grafos dirigidos.
- Mostrar pesos en aristas.
- Resaltar recorridos y resultados de algoritmos.
- Mostrar ayuda y mensajes didácticos.

Capa de rutas Flask

Responsable de:

- Recibir solicitudes HTTP.
- Validar datos básicos.
- Invocar servicios.
- Retornar HTML o JSON.

No debe contener lógica interna de grafos ni algoritmos.

Capa de servicios

Responsable de:

- Coordinar operaciones.
- Manejar estado de sesión.
- Capturar excepciones.
- Traducir respuestas a mensajes didácticos.
- Entregar estados serializables al frontend.

Capa de adaptadores

Responsable de normalizar el acceso a los TAD del módulo.

Adaptadores previstos:

- GraphAdapter.
- UnionFindAdapter.

Capa de dominio

Contiene los TAD originales:

- `grafo.py`.
- `union_find.py`.
- Dependencias reutilizadas: `cola.py`, `pila.py`, `cola_prioridad.py`, `exceptions.py`.

Esta capa debe permanecer independiente de Flask, plantillas y JavaScript.

8. Estructura de carpetas propuesta

```
visualizador_estructuras/  
|  
├─ app/  
|   └─ routes/  
|       └─ graph_routes.py  
|       └─ help_routes.py  
|  
|  
|  
|  
|
```

```

├── services/
│   ├── graph_service.py
│   ├── session_service.py
│   └── help_service.py
├── adapters/
│   ├── base_adapter.py
│   ├── graph_adapter.py
│   └── union_find_adapter.py
├── domain/
│   ├── graphs/
│   │   ├── __init__.py
│   │   ├── grafo.py
│   │   └── union_find.py
│   └── common/
│       ├── cola.py
│       ├── pila.py
│       ├── cola_prioridad.py
│       └── exceptions.py
├── templates/
│   ├── base.html
│   ├── partials/
│   │   ├── header.html
│   │   ├── footer.html
│   │   ├── navbar.html
│   │   ├── messages.html
│   │   └── help_panel.html
│   ├── pages/
│   │   ├── graphs.html
│   │   └── graph_view.html
│   └── help/
│       ├── graphs_help.html
│       ├── bfs_help.html
│       ├── dfs_help.html
│       ├── dijkstra_help.html
│       ├── bellman_ford_help.html
│       ├── prim_help.html
│       ├── kruskal_help.html
│       └── union_find_help.html
└── static/
    ├── css/
    │   ├── graphs.css
    │   └── graph_visualizer.css

```

```

├── js/
│   ├── graph_api_client.js
│   ├── graph_visualizer.js
│   ├── graph_algorithms_view.js
│   ├── graph_animations.js
│   └── graph_helpers.js
└── img/

├── tests/
│   ├── test_graph_adapter.py
│   ├── test_union_find_adapter.py
│   ├── test_graph_routes.py
│   ├── test_graph_visual_state.py
│   ├── test_graph_algorithms.py
│   └── test_graph_help.py
└── docs/
    ├── graphs_sdd.md
    ├── graphs_user_guide.md
    └── future_c_integration.md

```

9. Contratos de interfaz UI/UX

Pantalla del módulo de grafos

Debe incluir:

- Título: "Grafos".
- Descripción general.
- Selector de tipo de grafo:
 - Dirigido.
 - No dirigido.
- Opción de peso:
 - Usar peso por defecto.
 - Usar peso explícito.
- Botón "Crear grafo".
- Botón "Ayuda del módulo".

Pantalla de visualización de grafo

Header / Navbar
Nombre del módulo / tipo de grafo
Descripción breve

Panel de operaciones y algoritmos	Visualización del grafo
Resultados: recorridos, caminos, MST	
Explicación paso a paso	
Resultado / errores	
Footer	

Contrato visual para grafo

```
{
  "structure": "graph",
  "directed": false,
  "weighted": true,
  "nodes": [
    {"id": "A", "label": "A", "highlighted": false, "visited_order": null},
    {"id": "B", "label": "B", "highlighted": true, "visited_order": 1}
  ],
  "edges": [
    {"id": "A-B", "source": "A", "target": "B", "weight": 4, "directed":
false, "highlighted": true}
  ],
  "metadata": {
    "vertices_count": 2,
    "edges_count": 1,
    "is_empty": false
  },
  "last_operation": {
    "name": "insertar_arista",
    "status": "success",
    "message": "La arista A-B fue insertada correctamente."
  }
}
```

Contrato visual para resultados de algoritmos

```
{
  "algorithm": "dijkstra",
  "start": "A",
  "distances": {
    "A": 0,
    "B": 4,
    "C": 7
  },
  "previous": {
    "A": null,
    "B": "A",
```

```

    "C": "B"
  },
  "highlighted_nodes": ["A", "B", "C"],
  "highlighted_edges": ["A-B", "B-C"],
  "message": "Se calcularon las distancias mínimas desde A."
}

```

10. Reglas de negocio

ID	Regla	Descripción
RB-G-001	Operaciones válidas	Solo se deben mostrar operaciones soportadas por el TAD seleccionado
RB-G-002	Vértice inexistente	No se deben ejecutar vecinos, peso, BFS, DFS, Dijkstra o Bellman-Ford sobre vértices inexistentes
RB-G-003	Dijkstra sin negativos	Dijkstra no debe ejecutarse si existe alguna arista con peso negativo
RB-G-004	Prim no dirigido	Prim solo debe ejecutarse en grafos no dirigidos
RB-G-005	Kruskal no dirigido	Kruskal solo debe ejecutarse en grafos no dirigidos
RB-G-006	Mensajes didácticos	Todo error técnico debe traducirse a lenguaje comprensible
RB-G-007	Dominio independiente	Los TAD no deben importar Flask ni depender de la interfaz
RB-G-008	Estado serializable	El backend debe entregar estados JSON serializables
RB-G-009	Contrato estable	Los nombres internos de operaciones deben mantenerse estables
RB-G-010	Integración futura con C	El cambio a C debe ocurrir detrás de adaptadores
RB-G-011	No inventar operaciones	Si una operación no existe en el TAD, debe marcarse como pendiente
RB-G-012	Validación de entradas	Todo vértice, arista y peso debe validarse antes de ejecutar operaciones
RB-G-013	Peso por defecto	Si no se ingresa peso, la arista debe usar peso 1
RB-G-014	Visualización dirigida	Los grafos dirigidos deben mostrar flechas

ID	Regla	Descripción
RB-G-015	Visualización no dirigida	Los grafos no dirigidos deben evitar duplicación visual de aristas
RB-G-016	Union-Find	Union-Find debe tratarse como estructura auxiliar del módulo de grafos

11. Operaciones iniciales por TAD

Grafo

Operaciones esperadas:

- `insertar_vertice(vertice)`.
- `eliminar_vertice(vertice)`.
- `insertar_arista(origen, destino, peso=1)`.
- `eliminar_arista(origen, destino)`.
- `existe_vertice(vertice)`.
- `existe_arista(origen, destino)`.
- `vertices()`.
- `vecinos(vertice)`.
- `peso(origen, destino)`.
- `bfs(inicio)`.
- `dfs(inicio)`.
- `dijkstra(inicio)`.
- `bellman_ford(inicio)`.
- `prim(inicio=None)`.
- `kruskal()`.
- `aristas()`.
- `cantidad_vertices()`.
- `cantidad_aristas()`.

Union-Find

Operaciones esperadas:

- `agregar(x)`.
- `buscar(x)` / `find(x)`.
- `unir(a, b)` / `union(a, b)`.
- `conectados(a, b)`.

Observación didáctica:

Union-Find debe visualizarse como apoyo conceptual de Kruskal. Si el TAD no expone directamente sus mapas internos de padres y rangos mediante métodos públicos, el adaptador no debe depender de atributos privados salvo que se documente como decisión técnica temporal. La opción preferida es exponer un estado visual controlado desde el adaptador o marcar la visualización interna como pendiente.

12. Especificación de algoritmos

Algoritmo general para ejecutar una operación de grafo

Entrada:

- Tipo de grafo.
- Operación solicitada.
- Parámetros del usuario.
- Estado actual de sesión.

Proceso:

1. Validar que exista un grafo activo.
2. Validar que la operación pertenezca al conjunto soportado.
3. Validar parámetros de entrada.
4. Recuperar instancia desde sesión.
5. Invocar el adaptador correspondiente.
6. Ejecutar el método real del TAD.
7. Capturar excepciones del dominio.
8. Traducir resultado a mensaje didáctico.
9. Generar estado visual serializable.
10. Retornar respuesta HTML o JSON a la UI.

Salida:

- Estado actualizado.
- Resultado de operación.
- Mensaje didáctico.
- Datos para visualización.

Algoritmo para renderizar grafo

Entrada: Estado serializado del grafo.

Proceso:

1. Limpiar contenedor visual.
2. Crear nodos a partir de la lista de vértices.
3. Crear aristas a partir de la lista de aristas.
4. Aplicar flechas si el grafo es dirigido.
5. Mostrar pesos si el grafo es ponderado.
6. Aplicar resaltados de nodos y aristas.
7. Mostrar metadatos: cantidad de vértices, aristas y tipo.

Salida: Grafo dibujado en pantalla.

Algoritmo didáctico para BFS

Entrada: Vértice inicial.

Proceso:

1. Validar que el vértice exista.
2. Ejecutar `bfs(inicio)`.
3. Obtener orden de visita.
4. Marcar nodos visitados según su orden.
5. Mostrar explicación del uso de cola.

Salida: Lista de vértices visitados y visualización resaltada.

Algoritmo didáctico para DFS

Entrada: Vértice inicial.

Proceso:

1. Validar que el vértice exista.
2. Ejecutar `dfs(inicio)`.
3. Obtener orden de visita.
4. Marcar nodos visitados según su orden.
5. Mostrar explicación del uso de pila.

Salida: Lista de vértices visitados y visualización resaltada.

Algoritmo didáctico para Dijkstra

Entrada: Vértice inicial.

Proceso:

1. Validar que el vértice exista.
2. Validar que no existan pesos negativos.
3. Ejecutar `dijkstra(inicio)`.
4. Mostrar distancias.
5. Mostrar predecesores.
6. Resaltar caminos resultantes cuando sea posible.

Salida: Distancias mínimas, predecesores y visualización resaltada.

Algoritmo didáctico para Bellman-Ford

Entrada: Vértice inicial.

Proceso:

1. Validar que el vértice exista.
2. Ejecutar `bellman_ford(inicio)`.
3. Mostrar distancias.
4. Mostrar predecesores.
5. Indicar si existe ciclo negativo.

Salida: Distancias, predecesores e indicador de ciclo negativo.

Algoritmo didáctico para Prim

Entrada: Vértice inicial opcional.

Proceso:

1. Validar que el grafo sea no dirigido.
2. Ejecutar `prim(inicio)`.
3. Mostrar aristas del árbol de expansión mínima.
4. Mostrar peso total.
5. Resaltar aristas seleccionadas.

Salida: Aristas MST, peso total y visualización resaltada.

Algoritmo didáctico para Kruskal

Entrada: Grafo no dirigido.

Proceso:

1. Validar que el grafo sea no dirigido.
2. Ejecutar `kruskal()`.
3. Mostrar aristas ordenadas conceptualmente por peso.
4. Mostrar aristas seleccionadas.
5. Mostrar peso total.
6. Explicar el uso de Union-Find para evitar ciclos.

Salida: Aristas MST, peso total y explicación didáctica.

13. API / servicios

Rutas web sugeridas

Endpoint	Método	Descripción
<code>/grafos</code>	GET	Página del módulo de grafos
<code>/grafos/visualizador</code>	GET	Página de visualización de grafo
<code>/api/grafos/create</code>	POST	Crear grafo dirigido o no dirigido
<code>/api/grafos/operation</code>	POST	Ejecutar operación sobre el grafo
<code>/api/grafos/state</code>	GET	Obtener estado actual serializado
<code>/api/grafos/reset</code>	POST	Reiniciar grafo
<code>/api/grafos/algorithm</code>	POST	Ejecutar algoritmo de grafo
<code>/ayuda/grafos</code>	GET	Ayuda general del módulo

Endpoint	Método	Descripción
/ayuda/grafos/<topic_id>	GET	Ayuda específica por operación o algoritmo

Servicio: GraphService

Responsabilidades:

- Crear grafos dirigidos o no dirigidos.
- Ejecutar operaciones básicas.
- Ejecutar algoritmos.
- Consultar estado.
- Reiniciar grafo.
- Generar mensajes didácticos.

Métodos conceptuales:

```
create_graph(directed=False, weighted=False)
execute_operation(operation, payload)
execute_algorithm(algorithm, payload)
get_graph_state()
reset_graph()
```

Servicio: SessionService

Responsabilidades:

- Guardar estado temporal.
- Recuperar grafo activo.
- Limpiar sesión.
- Serializar y deserializar datos.

Servicio: HelpService

Responsabilidades:

- Obtener ayuda del módulo.
- Obtener ayuda por algoritmo.
- Obtener descripción de operaciones.
- Proveer complejidades y errores comunes.

Contrato de adaptador base

Cada adaptador debe cumplir una interfaz común:

```
create()
execute(operation_name, payload)
to_visual_state()
```

```
reset()
get_supported_operations()
```

Contrato de GraphAdapter

```
create(directed=False)
insert_vertex(vertex)
delete_vertex(vertex)
insert_edge(source, target, weight=1)
delete_edge(source, target)
run_bfs(start)
run_dfs(start)
run_dijkstra(start)
run_bellman_ford(start)
run_prim(start=None)
run_kruskal()
to_visual_state()
reset()
get_supported_operations()
```

14. Trazabilidad

RF	Caso de uso	Módulo	Prueba
RF-G-001	CU-G-001	Graph Routes	test_graph_module_loads
RF-G-002	CU-G-002	GraphService	test_create_directed_and_undirected_graph
RF-G-003	CU-G-002, CU-G-004	GraphAdapter	test_weighted_edges
RF-G-004	CU-G-003	GraphAdapter	test_insert_vertex
RF-G-005	CU-G-005	GraphAdapter	test_delete_vertex
RF-G-006	CU-G-004	GraphAdapter	test_insert_edge
RF-G-007	CU-G-006	GraphAdapter	test_delete_edge
RF-G-008	CU-G-003 a CU-G-006	GraphAdapter	test_vertices_list
RF-G-009	CU-G-004 a CU-G-006	GraphAdapter	test_edges_list

RF	Caso de uso	Módulo	Prueba
RF-G-010	CU-G-003 a CU-G-006	GraphAdapter	test_neighbors
RF-G-011	CU-G-004	GraphAdapter	test_edge_weight
RF-G-012	CU-G-007	GraphAdapter	test_bfs
RF-G-013	CU-G-008	GraphAdapter	test_dfs
RF-G-014	CU-G-009	GraphAdapter	test_dijkstra
RF-G-015	CU-G-010	GraphAdapter	test_bellman_ford
RF-G-016	CU-G-011	GraphAdapter	test_prim
RF-G-017	CU-G-012	GraphAdapter / UnionFindAdapter	test_kruskal
RF-G-018	Todos	Graph Visualizer JS	test_graph_visual_state
RF-G-019	CU-G-007 a CU-G-012	Graph Visualizer JS	test_algorithm_highlights
RF-G-020	CU-G-012	UnionFindAdapter	test_union_find_visual_state
RF-G-021	CU-G-013	HelpService	test_graph_help_available
RF-G-022	CU-G-013	HelpService	test_algorithm_help_available
RF-G-023	CU-G-003 a CU-G-012	Services / Exceptions	test_graph_error_messages
RF-G-024	CU-G-002 a CU-G-012	SessionService	test_graph_session_persistence
RF-G-025	Todos	Domain / Services / Adapters	test_no_flask_dependency_in_graph_domain
RF-G-026	Futuro	Adapter Layer	test_graph_adapter_contract_stability
RF-G-027	Todos	Templates	test_base_template_reuse

15. Plan de pruebas

Pruebas unitarias

Deben cubrir:

- GraphAdapter.
- UnionFindAdapter.
- GraphService.
- Conversión a estado visual.
- Manejo de errores.
- Validaciones de entrada.
- Operaciones básicas del grafo.
- Algoritmos de grafos.

Pruebas de integración

Deben validar:

- Rutas Flask del módulo de grafos.
- Creación de grafo dirigido y no dirigido.
- Inserción y eliminación de vértices.
- Inserción y eliminación de aristas.
- Ejecución de algoritmos.
- Respuesta JSON/HTML.
- Persistencia temporal en sesión.

Pruebas de interfaz

Deben validar:

- Carga de plantillas.
- Botones visibles.
- Visualización de vértices.
- Visualización de aristas.
- Visualización de flechas en grafos dirigidos.
- Visualización de pesos.
- Resaltado de recorridos y resultados.
- Mensajes de ayuda.

Pruebas didácticas

Deben validar que:

- Cada operación tenga explicación.
- Cada algoritmo tenga explicación.
- Los errores sean comprensibles.
- Las restricciones de algoritmos estén claras.
- Union-Find se explique como apoyo de Kruskal.

16. Criterios de aceptación

La primera versión del módulo de grafos será aceptada cuando:

1. La aplicación Flask mantenga la navegación general.
 2. Exista una página del módulo de grafos.
 3. Se pueda crear un grafo dirigido o no dirigido.
 4. Se puedan insertar y eliminar vértices.
 5. Se puedan insertar y eliminar aristas.
 6. Se puedan mostrar vértices, aristas, vecinos y pesos.
 7. Se pueda ejecutar BFS desde un vértice válido.
 8. Se pueda ejecutar DFS desde un vértice válido.
 9. Se pueda ejecutar Dijkstra con pesos no negativos.
 10. Se bloquee Dijkstra con pesos negativos y se explique el motivo.
 11. Se pueda ejecutar Bellman-Ford y detectar ciclo negativo si aplica.
 12. Se pueda ejecutar Prim en grafos no dirigidos.
 13. Se pueda ejecutar Kruskal en grafos no dirigidos.
 14. Union-Find esté integrado como estructura auxiliar del módulo.
 15. La visualización muestre nodos, aristas, pesos y direcciones.
 16. Los errores sean controlados y explicados de forma didáctica.
 17. El código de dominio no importe Flask.
 18. Las plantillas reutilicen header, footer y navbar del proyecto.
 19. Exista ayuda didáctica del módulo y de cada algoritmo.
 20. El código Python cumpla PEP8 y tenga docstrings.
 21. El frontend esté separado en archivos CSS y JavaScript.
 22. Existan pruebas iniciales del módulo.
 23. El proyecto quede preparado para futuras implementaciones de TAD en C.
-

17. Gestión del proyecto

Metodología

Desarrollo basado en especificaciones, con implementación incremental por operaciones y algoritmos.

Herramientas sugeridas

- Python 3.
- Flask.
- Pytest.
- HTML5.
- CSS3.
- JavaScript.
- Git.
- Agente IA de desarrollo.

Fases

Fase 1 — Análisis de TAD de grafos

- Revisar `grafo.py` y `union_find.py`.
- Extraer clases, métodos, alias y excepciones reales.
- Identificar dependencias: cola, pila, cola de prioridad y excepciones.
- Identificar limitaciones de visualización.

Fase 2 — Base del módulo

- Crear rutas de grafos.
- Crear plantillas del módulo.
- Integrar navegación desde la página principal.

Fase 3 — Adaptadores

- Crear GraphAdapter.
- Crear UnionFindAdapter.
- Normalizar operaciones.
- Crear estado visual serializable.

Fase 4 — Operaciones básicas

- Crear grafo.
- Insertar vértices.
- Eliminar vértices.
- Insertar aristas.
- Eliminar aristas.
- Consultar vecinos, pesos, vértices y aristas.

Fase 5 — Algoritmos

- Implementar integración visual de BFS.
- Implementar integración visual de DFS.
- Implementar integración visual de Dijkstra.
- Implementar integración visual de Bellman-Ford.
- Implementar integración visual de Prim.
- Implementar integración visual de Kruskal y Union-Find.

Fase 6 — Visualización

- Crear visualizador de grafos.
- Dibujar nodos, aristas, flechas y pesos.
- Crear resaltado de recorridos y resultados.
- Crear panel de resultados de algoritmos.

Fase 7 — Ayuda didáctica

- Crear ayuda general del módulo.
- Crear ayuda para cada algoritmo.
- Crear ayuda para Union-Find.
- Incluir complejidad, casos de uso, errores comunes y ejemplos.

Fase 8 — Pruebas y documentación

- Agregar pruebas unitarias.
- Agregar pruebas de rutas.
- Documentar arquitectura.
- Documentar uso del módulo.
- Documentar preparación para C.

18. Integración con agentes de IA

18.1 Roles

Agente	Responsabilidad
Agente arquitecto	Mantener arquitectura, trazabilidad y separación de capas
Agente backend	Implementar Flask, servicios, adaptadores, sesión y pruebas backend
Agente frontend	Implementar plantillas, CSS, JS y visualizaciones de grafos
Agente documental	Crear ayudas didácticas, README, guías y docstrings
Agente QA	Crear pruebas unitarias, integración y validación de aceptación

18.2 Prompt base para agente desarrollador

Actúa como un agente desarrollador senior especializado en Python, Flask, HTML5, CSS3, JavaScript y arquitectura limpia para aplicaciones educativas.

Contexto del proyecto:

Debes construir el tercer módulo de una aplicación web didáctica llamada "Visualizador Web de Estructuras de Datos". El objetivo del módulo es permitir que estudiantes aprendan grafos mediante visualizaciones interactivas.

La aplicación general tiene tres módulos:

1. Estructuras secuenciales.
2. Estructuras jerárquicas.
3. Grafos.

En esta fase solo debes implementar el módulo de grafos.

TAD disponibles inicialmente para este módulo:

- grafo.py
- union_find.py
- exceptions.py

Dependencias de dominio reutilizadas por el TAD Grafo:

- cola.py
- pila.py
- cola_prioridad.py

Objetivo técnico:

Crear un módulo Flask modular que permita crear grafos dirigidos o no dirigidos, ejecutar operaciones básicas, visualizar el estado del grafo y ejecutar algoritmos como BFS, DFS, Dijkstra, Bellman-Ford, Prim y Kruskal. La lógica de los TAD debe estar completamente separada de la capa visual.

Restricciones obligatorias:

1. No mezclar lógica de grafos con rutas Flask ni plantillas HTML.
2. No modificar los TAD originales salvo que sea estrictamente necesario y documentado.
3. No inventar operaciones que no existan en los TAD. Si una operación requerida no existe, marcarla como pendiente.
4. Todo código Python debe cumplir PEP8.
5. Todo módulo, clase y función Python debe tener docstrings.
6. Usar nombres claros y type hints cuando sea razonable.
7. No usar eval.
8. Validar entradas de usuario.
9. Todos los errores deben traducirse a mensajes didácticos.
10. La interfaz debe usar plantillas reutilizables con header, footer, navbar y layout base.
11. El frontend debe estar separado en HTML, CSS y JavaScript.
12. La arquitectura debe quedar preparada para que en una fase futura los TAD puedan implementarse en C.
13. No implementar algoritmos de grafos que no existan en el TAD.
14. No exponer directamente objetos internos no serializables al frontend.
15. Dijkstra no debe ejecutarse si existen pesos negativos.
16. Prim y Kruskal solo deben ejecutarse en grafos no dirigidos.

Arquitectura esperada:

Implementa una arquitectura por capas:

- Capa de presentación:
 - templates/
 - static/css/
 - static/js/
- Capa de rutas:
 - app/routes/graph_routes.py
 - app/routes/help_routes.py
- Capa de servicios:
 - app/services/graph_service.py
 - app/services/session_service.py
 - app/services/help_service.py
- Capa de adaptadores:
 - app/adapters/base_adapter.py
 - app/adapters/graph_adapter.py
 - app/adapters/union_find_adapter.py

- Capa de dominio:
 - app/domain/graphs/
 - Allí deben ubicarse grafo.py y union_find.py.

Funcionalidades mínimas:

1. Página del módulo de grafos.
2. Navegación desde la página principal.
3. Crear grafo dirigido o no dirigido.
4. Insertar vértices.
5. Eliminar vértices.
6. Insertar aristas con peso opcional.
7. Eliminar aristas.
8. Consultar vértices, aristas, vecinos y pesos.
9. Ejecutar BFS.
10. Ejecutar DFS.
11. Ejecutar Dijkstra.
12. Ejecutar Bellman-Ford.
13. Ejecutar Prim.
14. Ejecutar Kruskal.
15. Visualizar nodos, aristas, pesos y dirección.
16. Resaltar recorridos y resultados.
17. Mostrar Union-Find como apoyo conceptual de Kruskal.
18. Mensajes didácticos después de cada operación.
19. Manejo de errores para vértices inexistentes, pesos negativos y algoritmos incompatibles.
20. Sección de ayuda general para grafos.
21. Ayuda específica para cada algoritmo.

Contrato de adaptador:

Cada adaptador debe implementar una interfaz común equivalente a:

- create()
- execute(operation_name, payload)
- to_visual_state()
- reset()
- get_supported_operations()

El método to_visual_state() debe devolver datos serializables, por ejemplo:

```
{
  "structure": "graph",
  "directed": false,
  "weighted": true,
  "nodes": [],
  "edges": [],
  "metadata": {
    "vertices_count": 0,
    "edges_count": 0,
    "is_empty": true
  },
  "last_operation": {
```

```

    "name": "create_graph",
    "status": "success",
    "message": "Grafo creado correctamente."
  }
}

```

Pruebas requeridas:

Crea pruebas para:

1. Carga del módulo de grafos.
2. Creación de grafo dirigido y no dirigido.
3. Inserción y eliminación de vértices.
4. Inserción y eliminación de aristas.
5. Consulta de vértices, aristas, vecinos y pesos.
6. Ejecución de BFS.
7. Ejecución de DFS.
8. Ejecución de Dijkstra.
9. Bloqueo didáctico de Dijkstra con pesos negativos.
10. Ejecución de Bellman-Ford.
11. Ejecución de Prim en grafo no dirigido.
12. Bloqueo didáctico de Prim en grafo dirigido.
13. Ejecución de Kruskal en grafo no dirigido.
14. Bloqueo didáctico de Kruskal en grafo dirigido.
15. Conversión a estado visual.
16. Persistencia temporal del estado en sesión.
17. Disponibilidad de ayuda didáctica.
18. Integración conceptual de Union-Find.

Criterios de aceptación:

La tarea se considera completa si:

1. La aplicación Flask ejecuta sin errores.
2. El módulo de grafos está disponible desde la navegación.
3. Se puede crear un grafo dirigido o no dirigido.
4. Las operaciones disponibles se obtienen desde adaptadores.
5. El estado visual se actualiza después de cada operación.
6. Los algoritmos disponibles se ejecutan según las restricciones del TAD.
7. El código de dominio no importa Flask.
8. Las plantillas reutilizan header, footer y navbar.
9. Existe ayuda didáctica del módulo y de cada algoritmo.
10. Existen pruebas iniciales.
11. El proyecto queda preparado para futura integración de TAD implementados en C.

Prohibiciones explícitas:

- No generar código monolítico.
- No poner lógica de grafos en las rutas Flask.
- No poner lógica de negocio en JavaScript si debe estar en backend.
- No usar eval.
- No ignorar excepciones.
- No inventar métodos inexistentes en los TAD.
- No eliminar archivos TAD originales.
- No acoplar la UI directamente a una clase concreta del dominio.

- No implementar algoritmos no disponibles en el TAD.
- No ejecutar Dijkstra con pesos negativos.
- No ejecutar Prim ni Kruskal en grafos dirigidos.

Formato de salida esperado:

1. Primero crea un plan de implementación detallado antes de desarrollar la aplicación.
2. El plan debe incluir:
 - análisis de los TAD suministrados;
 - mapa entre operaciones de UI y métodos reales;
 - estructura final de carpetas;
 - fases de implementación;
 - componentes backend, frontend, servicios, adaptadores y pruebas;
 - riesgos técnicos;
 - operaciones pendientes o no soportadas;
 - criterios de validación antes de codificar.
3. Después de presentar el plan, desarrolla la aplicación siguiendo ese plan.
4. Muestra el árbol de carpetas creado.
5. Explica brevemente la responsabilidad de cada carpeta.
6. Implementa los archivos necesarios.
7. Incluye comentarios y docstrings.
8. Incluye pruebas.
9. Incluye instrucciones para ejecutar:
 - creación de entorno virtual;
 - instalación de dependencias;
 - ejecución de Flask;
 - ejecución de pruebas.
10. Indica claramente cualquier operación pendiente porque no exista en los TAD suministrados.

Antes de codificar:

Analiza los TAD suministrados y extrae sus clases, métodos y excepciones reales. Usa esa información para construir los adaptadores. Si existe discrepancia entre esta especificación y los TAD reales, respeta los TAD reales y documenta la diferencia.

19. Control de versiones

Rama principal

main

Debe contener solo versiones estables.

Rama de desarrollo

develop

Para integración de funcionalidades.

Ramas sugeridas

```
feature/graphs-module
feature/graph-routes
feature/graph-adapter
feature/union-find-adapter
feature/graph-visualizer
feature/graph-algorithms
feature/graph-help
feature/graph-tests
```

Convención de commits

```
feat: agrega módulo de grafos
feat: agrega adaptador de grafo
feat: agrega visualizador de grafos
feat: integra BFS y DFS
feat: integra Dijkstra y Bellman-Ford
feat: integra Prim y Kruskal
fix: corrige validación de vértice inexistente
docs: agrega ayuda didáctica para Kruskal
test: agrega pruebas para algoritmos de grafos
refactor: separa visualización de operaciones de grafo
```

20. Decisiones arquitectónicas importantes

ID	Decisión	Justificación
DA-G-001	Separar dominio y presentación	Evita acoplamiento entre TAD y Flask
DA-G-002	Usar adaptadores	Permite normalizar operaciones y preparar integración con C
DA-G-003	Serializar estado visual	Facilita comunicación backend/frontend
DA-G-004	Usar plantillas base	Mantiene consistencia con módulos anteriores

ID	Decisión	Justificación
DA-G-005	Union-Find pertenece a grafos	Se usa como soporte conceptual de Kruskal
DA-G-006	Respetar restricciones algorítmicas	Evita ejecutar algoritmos fuera de su dominio válido
DA-G-007	Mantener TAD originales	Reduce riesgo de alterar implementaciones existentes
DA-G-008	Marcar pendientes técnicos	Evita que agentes IA inventen operaciones o datos no disponibles
DA-G-009	Preparar integración con C	Permite cambiar implementación sin rediseñar la UI